

# **Rapport S1.02**

## **SpellWar**

### **Evaluation Finale**

# Sommaire :

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| <b>0. - Spécification externes.....</b>                               | <b>3</b>  |
| 0.1 - Les entités :.....                                              | 3         |
| 0.2 - Saisie du joueur :.....                                         | 3         |
| 0.3 - Fin de partie :.....                                            | 3         |
| 0.4 - Déplacements invalides :.....                                   | 4         |
| <b>1. - Spécifications internes.....</b>                              | <b>5</b>  |
| 1.1 - Représentation des informations :.....                          | 5         |
| 1.2 - Interactions/Collisions entre entités de la tableau.....        | 6         |
| 1.3 - Dictionnaire des données.....                                   | 7         |
| <b>2. - Algorithmes de programmes.....</b>                            | <b>8</b>  |
| 2.1 - Les étapes ordonnées du jeu.....                                | 8         |
| 2.2 - Les étapes redondantes du programme.....                        | 8         |
| <b>3. Jeux d'essais.....</b>                                          | <b>9</b>  |
| <b>4. Algorithmes.....</b>                                            | <b>10</b> |
| 4.1 - Algorithme Principal.....                                       | 10        |
| 4.1.1 - Algorithme de l'initialisation.....                           | 18        |
| 4.1.2 - Algorithme du déroulement du jeu.....                         | 20        |
| 4.1.3 - Algorithme de fin de partie.....                              | 22        |
| 4.2 - Algorithmes des sous-programmes.....                            | 24        |
| 4.2.1 - afficherZoneDeJeu.....                                        | 24        |
| 4.2.2 - generer.....                                                  | 26        |
| 4.2.3 - avoirCoordonnees.....                                         | 29        |
| 4.2.4 - disparaitre(coordonnées).....                                 | 31        |
| 4.2.5 - disparaitre(TypeEntite).....                                  | 34        |
| <b>5. Tests réalisés et traces d'exécution.....</b>                   | <b>39</b> |
| <b>6. Remarques.....</b>                                              | <b>40</b> |
| <b>7. Code C++.....</b>                                               | <b>41</b> |
| 7.1 - Choix d'organisation des fichiers composant le code source..... | 41        |
| 7.2 - Code source.....                                                | 43        |
| <b>8. Annexes.....</b>                                                | <b>44</b> |
| Annexe 1 – maquettes d'écran prévues dans les spécifications.....     | 44        |
| Annexe 2 – Traces d'exécution - copies d'écran.....                   | 45        |

## 0. - Spécification externes

### 0.1 - Les entités :

- Joueur : Se déplace ou lance un éclair en suivant les directives de l'utilisateur. Le joueur peut se déplacer verticalement. Le joueur ne peut saisir que 5 actions au maximum à la fois.
- Arcaflamme : Se déplacent verticalement et lancent des flammes. Les actions sont aléatoires.
- Nécrogriffe : Se déplacent aléatoirement sur la zone de jeu.
- Mur : Délimitent la zone de jeu à gauche. Ils bloquent le déplacement des entités au-delà de ceux-ci.
- Plafond : Délimitent la zone de jeu en haut et en bas. Ils bloquent le déplacement des entités au-delà de ceux-ci.
- Eclair : Ils sont lancés par le joueur. Se déplacent de droite à gauche.
- Flamme : Elles sont lancées par les Arcaflammes. Elles se déplacent de droite à gauche.
- Vide : Représente les zones vides de la zone de jeu.

### 0.2 - Saisie du joueur :

Le joueur peut saisir jusqu'à 10 actions en une saisie, au-delà de ce nombre, il n'y a pas d'intérêt pour le joueur de prévoir autant d'actions. Les actions possibles :

- m : Monter
- d : Descendre
- e : Jeter un éclair
- a : abandonner

### 0.3 - Fin de partie :

La partie prend fin si l'une de ces trois situations se produisent :

- Abandon du joueur
- Mort du joueur
- Mort de tous les ennemis

#### 0.4 - Déplacements invalides :

Si plusieurs ennemis se déplacent aux mêmes endroits, alors les actions de ces entités sont annulées. (ex: si un nécrogriffe se déplace sur un nécrogriffe ou un arcaflamme, son déplacement est annulé et il passe son tour)

Si le joueur entre un déplacement inexistant, il passera également son tour.

Si une entité souhaite monter/descendre alors qu'il y a un mur/plafond, alors le déplacement est annulé; l'entité passe son tour.

Si un nécrogriffe entre en collision avec une flamme, alors il devra faire un autre mouvement qui ne le fera pas percuter la flamme.

Il n'est pas possible d'avoir deux éclairs simultanément.

En effet, lors de la création de Spellwar, nous avons opté pour une approche basée sur la recherche de la première occurrence, ce qui exclut la possibilité de maintenir deux éclairs côte à côte.

## 1. - Spécifications internes

### 1.1 - Représentation des informations :

Zone de jeu : Représentée par un tableau de type Entité d'une longueur et largeur de 15 cases par défaut. Elle contiendra toutes les entités de la zone de jeu. Elle sera affichée. Chaque case représente une coordonnée, nous pouvons donc manipuler l'emplacement de chaque entité directement via le tableau.

#### Type d'entités :

```
enum TypeEntite
{
    Joueur,
    Arcaflamme,
    Necrogriffe,
    Flamme,
    Eclair,
    Mur,
    Plafond,
    Vide
};
```

#### Les couleurs :

```
enum Couleur
{
    gris = 7,
    bleu = 9,
    vert = 10,
    cyan = 11,
    rouge = 12,
    violet = 13,
    jaune = 14,
    blanc = 15
};
```

*\*Importés de la bibliothèque "gametools"*

Entités :

```
struct Entite
```

```
{
```

```
    TypeEntite type;        //Type qu'une entité peut prendre
```

```
    char representation;    //Représentation sur la zone de jeu
```

```
    Couleur couleur;        //Couleur de l'entité sur la zone de jeu
```

```
    bool dejaJoue;         // Savoir si l'entité a déjà joué
```

```
};
```

## 1.2 - Interactions/Collisions entre entités de la tableau

|                 | Arcaflammes    | Nécrogriffes          | Joueurs        | Projectiles (J)      | Projectiles (A)      | Mur vertical | Plafond |
|-----------------|----------------|-----------------------|----------------|----------------------|----------------------|--------------|---------|
| Arcaflammes     | Action annulée |                       |                |                      |                      |              |         |
| Nécrogriffes    | Action annulée | Action annulée        |                |                      |                      |              |         |
| Joueur          | X              | Le joueur perd        | X              |                      |                      |              |         |
| Projectiles (J) | Disparaît      | Disparaît             | X              | X                    |                      |              |         |
| Projectiles (A) | X              | Projectile disparaît  | Le joueur perd | Disparaît            | X                    |              |         |
| Mur vertical    | X              | Nécrogriffe disparaît | X              | Projectile disparaît | Projectile disparaît | X            |         |
| Plafond         | Action annulée | Action annulée        | Action annulée | X                    | X                    | X            | X       |

## 1.3 - Dictionnaire des données

| Nom                   | Type                | Signification                                               |
|-----------------------|---------------------|-------------------------------------------------------------|
| Arcaflamme            | Entite              | Donnée représentant les aracaflammes                        |
| Joueur                | Entite              | Donnée représentant le joueur                               |
| Necrogriffe           | Entite              | Donnée représentant les necrogriffes                        |
| Flamme                | Entite              | Donnée représentant les flammes                             |
| Eclair                | Entite              | Donnée représentant les eclairs                             |
| Mur                   | Entite              | Donnée représentant les mur                                 |
| Plafond               | Entite              | Donnée représentant les plafonds                            |
| Vide                  | Entite              | Donnée représentant les cases vides                         |
| zoneDeJeu             | Entite              | Tableau contenant tous les éléments du jeu                  |
| nbArcaflamme          | Entier              | Nombre d'arcaflamme qui seront placés dans la zone de jeu   |
| nbNecrogriffe         | Entier              | Nombre de necrogriffe qui seront placés dans la zone de jeu |
| instuctionsTour       | Chaine de carcatère | Saisie des actions du joueur                                |
| indiceLettreLue       | entier non signé    | Emplacement de la lettre qui est lue dans instructionTour   |
| tailleInstructionTour | entier non signé    | Taille de la saisie de l'utilisateur                        |
| ord                   | entier non signé    | Ordonnée d'une entité la tableau zoneDeJeu                  |
| abs                   | entier non signé    | Absice d'une entité la tableau zoneDeJeu                    |
| finDePartie           | char                | Permet de connaitre la ause de la fin de partie             |
| NOMBRE_ENNEMI         | entier non signé    | Nombre d'ennemis dans la zone de jeu                        |
| HAUTEUR               | entier non signé    | Hauteur de la zone de jeu                                   |
| LARGEUR               | entier non signé    | Largeur de la zone de jeu                                   |
| ORD_JOUEUR            | entier non signé    | Position de départ en ordonnées du joueur                   |
| ABS_JOUEUR            | entier non signé    | Position de départ en abscisses du joueur                   |

## 2. - Algorithmes de programmes

### 2.1 - Les étapes ordonnées du jeu

- Afficher la zone de jeu
- Remplir la zone de jeu
- Envoi d'une saisie à l'utilisateur
- Déplacement du joueur en fonction de la saisie
- Déplacement ennemi ou élément ou spécifique
- Vérifications des actions et positions des entités
  - Vérifier si l'ennemi est mort (bool "estMort")
  - Vérifier les positions ennemis (renvoyer un bool pour savoir si action faisable)
- Vérifications des conditions de fin de jeu
  - Vérifier combien il y a d'ennemis
  - Vérifier si le joueur est mort
- Si le jeu n'est pas terminé, la partie continue
- SINON le jeu s'arrête avec un message de victoire/défaite

### 2.2 - Les étapes redondantes du programme

- Ajouter une entité
- Savoir si une entité est morte
- Afficher la zone de jeu
- Connaître les coordonnées d'une entité dans la zone de jeu
- Changer les coordonnées d'une entité
- Définir les déplacement des entités
- Vérifier si un déplacement est possible
- Vérifier le nombre d'ennemis dans la zone de jeu
- Déplacer une entité dans la zone de jeu



### 3. Jeux d'essais

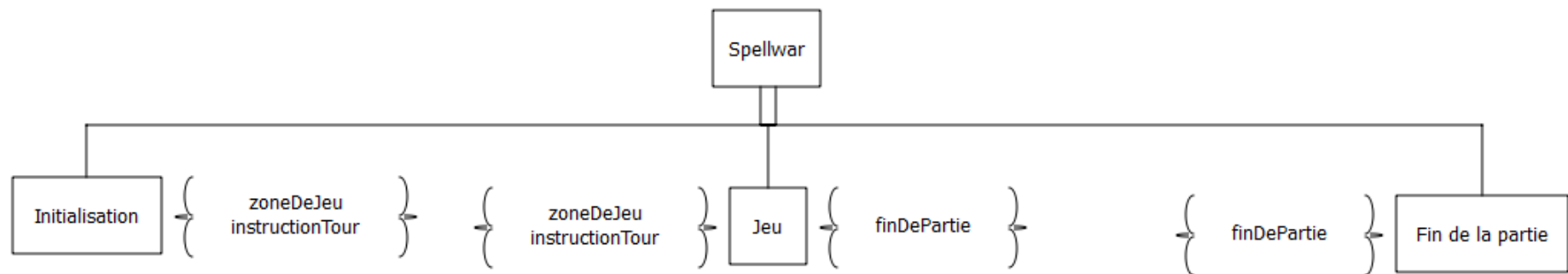
Scénarios qui seront testés.

Pour chaque scénario, liste des valeurs entrées et comportement attendu

|   | A                        | B       | C       | D | E                               | F                 |
|---|--------------------------|---------|---------|---|---------------------------------|-------------------|
| 1 | Tests programme Spellwar |         |         |   |                                 |                   |
| 2 | Données                  |         |         |   | Resultats attendus              | Affichage écran   |
| 3 | NOMBRE_ENNEMIS           | HAUTEUR | LARGEUR |   | /                               | /                 |
| 4 | 3                        | 15      | 15      |   | Le jeu se comporte normalement. | Rien d'anormal    |
| 5 | 2                        | 20      | 20      |   | Le jeu se comporte normalement. | Rien d'anormal    |
| 6 | 4                        | 30      | 30      |   | Le jeu se comporte normalement. | Rien d'anormal    |
| 7 | 5                        | 10      | 10      |   | Le jeu se comporte normalement. | Rien d'anormal    |
| 8 | 6                        | 5       | 5       |   | Le jeu ne se lance pas.         | Rien ne s'affiche |
| 9 | 6                        | 15      | 15      |   | Le jeu se comporte normalement. | Rien d'anormal    |

## 4. Algorithmes

### 4.1 - Algorithme Principal



**Description :** L'algorithme principal du jeu se décompose en 3 sous-problèmes principaux :

Initialiser les variables de début de partie : Dans cette étape, il s'agit d'initialiser les éléments permettant de commencer la partie. Dans notre cas, il s'agit d'initialiser la zone de jeu, ainsi que sa largeur, sa longueur, le nombre d'ennemis et le nombre de types d'entités.

Jouer la partie : Dans cette étape, il s'agit de placer, ajouter, déplacer les entités dans la zone de jeu. Le joueur contrôle via une saisie les déplacements de son personnage.

Fin de partie : Dans cette dernière étape, il s'agit de conclure la partie en présentant au joueur la raison pour laquelle la partie a pris fin :

- Victoire en battant tous les ennemis
- Défaite en se faisant toucher par un ennemi ou une flamme
- Défaite par abandon

## Dictionnaire des éléments associés à cet algorithme :

| Nom                   | Type                | Signification                                               |
|-----------------------|---------------------|-------------------------------------------------------------|
| Arcaflamme            | Entite              | Donnée représentant les aracaflammes                        |
| Joueur                | Entite              | Donnée représentant le joueur                               |
| Necrogriffe           | Entite              | Donnée représentant les necrogriffes                        |
| Flamme                | Entite              | Donnée représentant les flammes                             |
| Eclair                | Entite              | Donnée représentant les eclairs                             |
| Mur                   | Entite              | Donnée représentant les mur                                 |
| Plafond               | Entite              | Donnée représentant les plafonds                            |
| Vide                  | Entite              | Donnée représentant les cases vides                         |
| zoneDeJeu             | Entite              | Tableau contenant tous les éléments du jeu                  |
| nbArcaflamme          | Entier              | Nombre d'arcaflamme qui seront placés dans la zone de jeu   |
| nbNecrogriffe         | Entier              | Nombre de necrogriffe qui seront placés dans la zone de jeu |
| instuctionsTour       | Chaine de caractere | Saisie des actions du joueur                                |
| indiceLettreLue       | entier non signé    | Emplacement de la lettre qui est lue dans instructionTour   |
| tailleInstructionTour | entier non signé    | Taille de la saisie de l'utilisateur                        |
| ord                   | entier non signé    | Ordonnée d'une entité la tableau zoneDeJeu                  |
| abs                   | entier non signé    | Abscise d'une entité la tableau zoneDeJeu                   |
| finDePartie           | char                | Permet de connaitre la ause de la fin de partie             |
| NOMBRE_ENNEMI         | entier non signé    | Nombre d'ennemis dans la zone de jeu                        |
| HAUTEUR               | entier non signé    | Hauteur de la zone de jeu                                   |
| LARGEUR               | entier non signé    | Largeur de la zone de jeu                                   |
| ORD_JOUEUR            | entier non signé    | Position de départ en ordonnées du joueur                   |
| ABS_JOUEUR            | entier non signé    | Position de départ en abscisses du joueur                   |

## Dictionnaire des actions (complexes) associés à cet algorithme :

| Nom Action            | But                                                                                                                     | Données                                                                                        | Résultats                                                                |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| initialiserZoneDeJeu  | Remplir la zone de jeu avec les entités (joueur, nécrogriffe, arcaflamme, mur, plafond)                                 | Entite zoneDeJeu[HAUTEUR][LARGEUR]                                                             | Entite zoneDeJeu[HAUTEUR][LARGEUR]                                       |
| generer               | Générer les caractéristiques de l'entité de "type"                                                                      | TypeEntite type                                                                                | Entite                                                                   |
| initialisationEntites | Initialisation des entités dans la zone de jeu                                                                          | Entite zoneDeJeu[HAUTEUR][LARGEUR]                                                             | Entite zoneDeJeu[HAUTEUR][LARGEUR]                                       |
| afficherZoneDeJeu     | Afficher la zone de jeu ligne par ligne                                                                                 | const Entite zoneDeJeu[HAUTEUR][LARGEUR]                                                       | Affichage a l'écran                                                      |
| actionJoueur          | Déplacer le joueur dans la zoneDeJeu en fonction de la lettre lue                                                       | string deplacement<br>Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned short int indiceLettreLue | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned short int indiceLettreLue |
| avoirCoordonnees      | Dans le cas où l'entité de "type" existe dans la zone de jeu, renvoie ses coordonnées et trouve = vrai                  | const Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>TypeEntite typeEntite                              | unsigned int ord<br>unsigned int abs<br>bool trouve                      |
| saisieUtilisateur     | Envoyer une saisie à l'utilisateur avec un nombre de caractères limités et renvoyer cette dernière sous forme de string |                                                                                                | string                                                                   |

|                            |                                                                                                                                                                                  |                                                                                      |                                      |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------|
| disparaitre                | Remplacer, dans la zone de jeu, l'entité mise en paramètre parametre par l'entité "vide". Utilisé dans le cas où on ne connaît pas les coordonnées de l'entité.                  | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>TypeEntite typeEntite                          | Entite zoneDeJeu[HAUTEUR][LARGEUR]   |
| disparaitre                | Remplacer, dans la zone de jeu, l'entité aux coordonnées d'abscisses et d'ordonnées définies par l'entité "vide". Utilisé dans le cas où on connaît les coordonnées de l'entité. | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned int abs<br>unsigned int ord           | Entite zoneDeJeu[HAUTEUR][LARGEUR]   |
| verifFinDePartie           | Vérifier dans la zone de jeu si la partie est terminée                                                                                                                           | const Entite<br>zoneDeJeu[HAUTEUR][LARGEUR]                                          | char                                 |
| actionsEntite              | Actions produites par les entités dans la zone de jeu, à chaque tour                                                                                                             | Entite zoneDeJeu[HAUTEUR][LARGEUR]                                                   | Entite zoneDeJeu[HAUTEUR][LARGEUR]   |
| genererActionEntite        | Créer une action aléatoire pour une entité de type "typeEntite" aux coordonnées spécifiées dans la zone de jeu.                                                                  | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>TypeEntite typeEntite                          | unsigned int ord<br>unsigned int abs |
| gererCollisionsNecrogriffe | Vérifier la présence, dans la zone de jeu, d'une entité autre que vide aux coordonnées futures (ordonnées et abscisses) d'un Necrogriffe.                                        | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned int futurOrd<br>unsigned int futurAbs | char                                 |

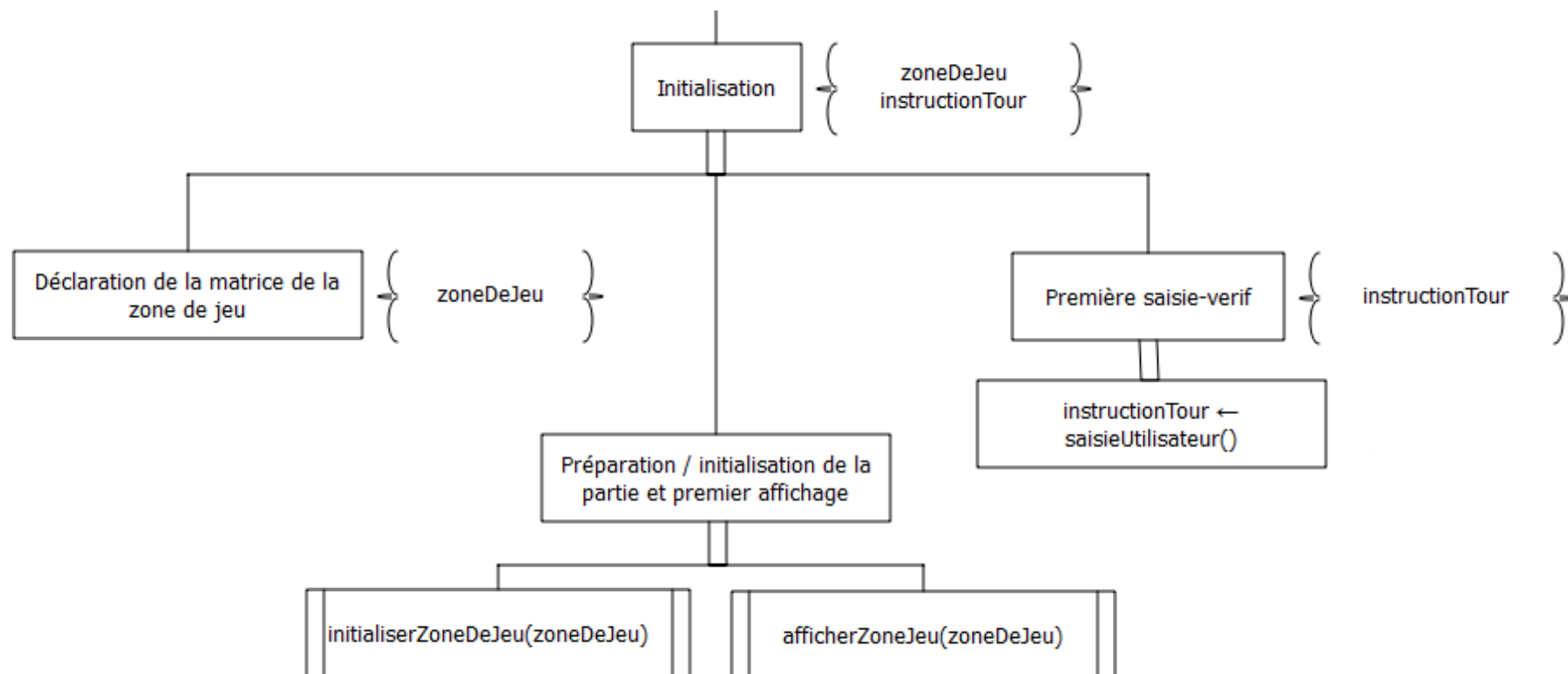
|                           |                                                                                                                                                     |                                                                                                                                      |                                    |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| gererCollisionsFlamme     | Vérifier la présence, dans la zone de jeu,<br>d'une entité autre que vide aux coordonnées futures (ordonnées et abscisses) d'une Flamme.            | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned int actuOrd<br>unsigned int actuAbs<br>unsigned int futurOrd<br>unsigned int futurAbs | char                               |
| gererCollisionsJoueur     | Vérifier la présence, dans la zone de jeu,<br>d'une entité autre que vide aux coordonnées futures (ordonnées et abscisses) d'un Joueur.             | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned int futurOrd<br>unsigned int futurAbs                                                 | char                               |
| gererCollisionsEclair     | Vérifier la présence, dans la zone de jeu,<br>d'une entité autre que vide aux coordonnées futures (ordonnées et abscisses) d'un Eclair.             | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned int actuOrd<br>unsigned int actuAbs<br>unsigned int futurOrd<br>unsigned int futurAbs | char                               |
| gererCollisionsArcaflamme | Vérifier la présence, dans la zone de jeu,<br>d'une entité autre que vide aux coordonnées futures (ordonnées et abscisses) d'un Arcaflamme.         | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned int futurOrd<br>unsigned int futurAbs                                                 | char                               |
| avancerProjectiles        | Faire avancer un projectile (typeEntite Flamme ou Eclair) dans la zone de jeu,<br>en renseignant ses coordonnées (ordonnées et abscisses) initiales | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>TypeEntite typeEntite<br>unsigned int ord<br>unsigned int abs                                  | Entite zoneDeJeu[HAUTEUR][LARGEUR] |

|                             |                                                                                                                                                                                                            |                                                                                                                                      |                                    |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| nouveauTourJoue             | Faire en sorte que chaque entité de la zone de jeu de la zone de jeu puisse rejouer                                                                                                                        | Entite zoneDeJeu[HAUTEUR][LARGEUR]                                                                                                   | Entite zoneDeJeu[HAUTEUR][LARGEUR] |
| switchCollisionsNecrogriffe | Switch permettant de traiter l'information renvoyée par le sous-programme gererCollisionsNecrogriffe().Il prend en arguments les coordonnées (ordonnées et abscisses) présentes et futures du nécrogriffe. | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned int actuOrd<br>unsigned int actuAbs<br>unsigned int futurOrd<br>unsigned int futurAbs | Entite zoneDeJeu[HAUTEUR][LARGEUR] |
| switchCollisionsArcaflamme  | Switch permettant de traiter l'information renvoyée par le sous-programme gererCollisionsArcaflamme().Il prend en arguments les coordonnées (ordonnées et abscisses) présentes et futures de l'arcaflame.  | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned int actuOrd<br>unsigned int actuAbs<br>unsigned int futurOrd<br>unsigned int futurAbs | Entite zoneDeJeu[HAUTEUR][LARGEUR] |
| switchCollisionsFlamme      | Switch permettant de traiter l'information renvoyée par le sous-programme gererCollisionsFlamme().Il prend en arguments les coordonnées (ordonnées et abscisses) présentes et futures de la flamme.        | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned int actuOrd<br>unsigned int actuAbs<br>unsigned int futurOrd<br>unsigned int futurAbs | Entite zoneDeJeu[HAUTEUR][LARGEUR] |



|                        |                                                                                                                                                                                                     |                                                                                                                                      |                                    |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| switchCollisionsEclair | Switch permettant de traiter l'information renvoyée par le sous-programme gererCollisionsEclair(). Il prend en arguments les coordonnées (ordonnées et abscisses) présentes et futures de l'éclair. | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned int actuOrd<br>unsigned int actuAbs<br>unsigned int futurOrd<br>unsigned int futurAbs | Entite zoneDeJeu[HAUTEUR][LARGEUR] |
| switchCollisionsJoueur | Switch permettant de traiter l'information renvoyée par le sous-programme gererCollisionsJoueur(). Il prend en arguments les coordonnées (ordonnées et abscisses) présentes et futures du joueur.   | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned int actuOrd<br>unsigned int actuAbs<br>unsigned int futurOrd<br>unsigned int futurAbs | Entite zoneDeJeu[HAUTEUR][LARGEUR] |

#### 4.1.1 - Algorithme de l'initialisation



**Description :** Cette partie du programme principal permet d'initialiser les éléments pour le bon déroulement du jeu:

- Déclaration de la tableau de la zone de jeu : Permet de créer la zone de jeu.
- Préparation de la partie et premier affichage : Initialise et crée la zone de jeu.
- Première saisie-verif : Effectue une première saisie-verif.

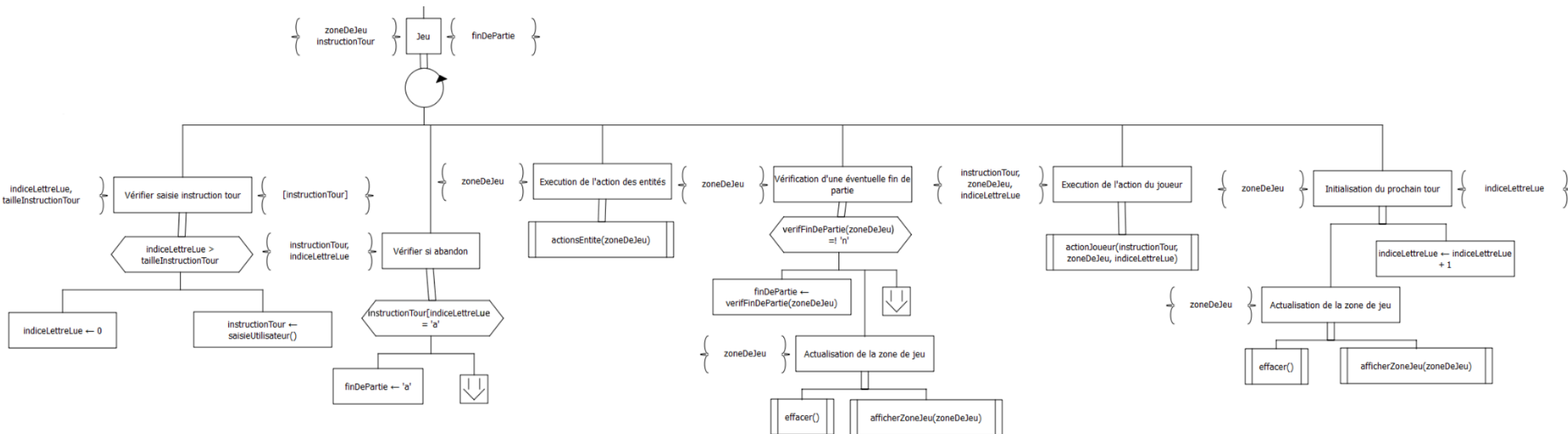
**Dictionnaire des éléments associés à cet algorithme :**

| Nom             | Type                 | Signification                                                  |
|-----------------|----------------------|----------------------------------------------------------------|
| zoneDeJeu       | Entite               | tableau contenant tous les éléments du jeu                     |
| instructionTour | Chaîne de caractères | Chaîne de caractères contenant la liste des actions à réaliser |

**Dictionnaire des actions (complexes) associés à cet algorithme :**

| Nom Action             | But                                                                                                                     | Données   | Résultats                                     |
|------------------------|-------------------------------------------------------------------------------------------------------------------------|-----------|-----------------------------------------------|
| initialiserZoneDeJeu() | Initialisation des entités dans la zone de jeu                                                                          | zoneDeJeu | Modification de la zone de jeu                |
| afficherZoneDeJeu()    | Afficher la zone de jeu ligne par ligne                                                                                 | zoneDeJeu | Affichage de la zone de jeu dans la console   |
| saisieUtilisateur()    | Envoyer une saisie à l'utilisateur avec un nombre de caractères limités et renvoyer cette dernière sous forme de string | /         | Chaîne de caractères saisie par l'utilisateur |

### 4.1.2 - Algorithme du déroulement du jeu



**Description :** Cette partie du programme principal veille au bon déroulement au sein du jeu. Il s'agit ici de vérifier la saisie d'instructionTour, de vérifier si le joueur abandonne, d'exécuter les actions des entités, de vérifier une éventuelle fin de partie, d'exécuter les actions du joueur et d'initialiser le début du prochain tour.

**Dictionnaire des éléments associés à cet algorithme :**

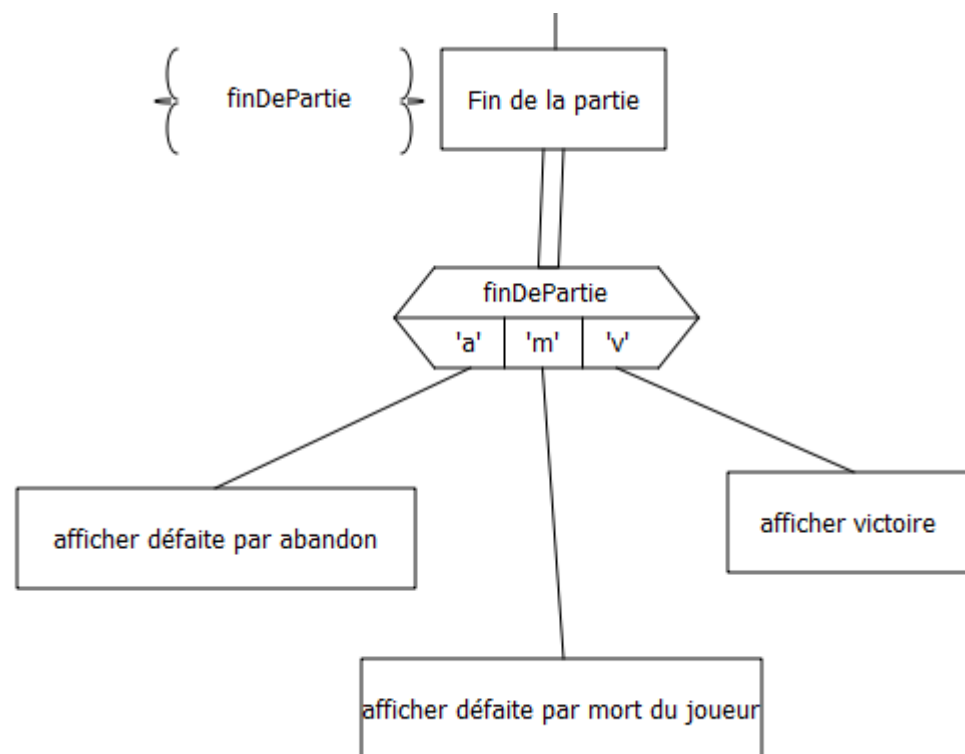
| Nom                   | Type                   | Signification                                             |
|-----------------------|------------------------|-----------------------------------------------------------|
| indiceLettreLue       | Entier court non signé | Emplacement de la lettre qui est lue dans instructionTour |
| tailleInstructionTour | Entier non signé       | Taille de la saisie de l'utilisateur                      |

**Dictionnaire des actions (complexes) associés à cet algorithme :**

| Nom Action        | But                                                                                                                     | Données                                                                                        | Résultats                                                                |
|-------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| afficherZoneDeJeu | Afficher la zone de jeu ligne par ligne                                                                                 | const Entite<br>zoneDeJeu[HAUTEUR][LARGEUR]                                                    | Affichage a l'écran                                                      |
| actionJoueur      | Déplacer le joueur dans la zonedejeu en fonction de la lettre lue                                                       | string deplacement<br>Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned short int indiceLettreLue | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned short int indiceLettreLue |
| saisieUtilisateur | Envoyer une saisie à l'utilisateur avec un nombre de caractères limités et renvoyer cette dernière sous forme de string |                                                                                                | string                                                                   |
| verifFinDePartie  | Vérifier dans la zone de jeu si la partie est terminée                                                                  | const Entite<br>zoneDeJeu[HAUTEUR][LARGEUR]                                                    | char                                                                     |
| actionsEntite     | Actions produites par les entités                                                                                       | Entite zoneDeJeu[HAUTEUR][LARGEUR]                                                             | Entite zoneDeJeu[HAUTEUR][LARGEUR]                                       |

|  |                                    |  |  |
|--|------------------------------------|--|--|
|  | dans la zone de jeu, à chaque tour |  |  |
|--|------------------------------------|--|--|

#### 4.1.3 - Algorithme de fin de partie



**Description :** Cette partie du programme principal nous permet de, à partir de l'indicateur finDePartie, définir la fin de partie et d'afficher un message en conséquence.

**Dictionnaire de variables :**

| Nom         | Type      | Signification                                                                                      |
|-------------|-----------|----------------------------------------------------------------------------------------------------|
| finDePartie | Caractère | Indicateur de fin de partie (n pour non fini, a pour abandon, m pour mort joueur, v pour victoire) |



## 4.2 - Algorithmes des sous-programmes

### 4.2.1 - afficherZoneDeJeu

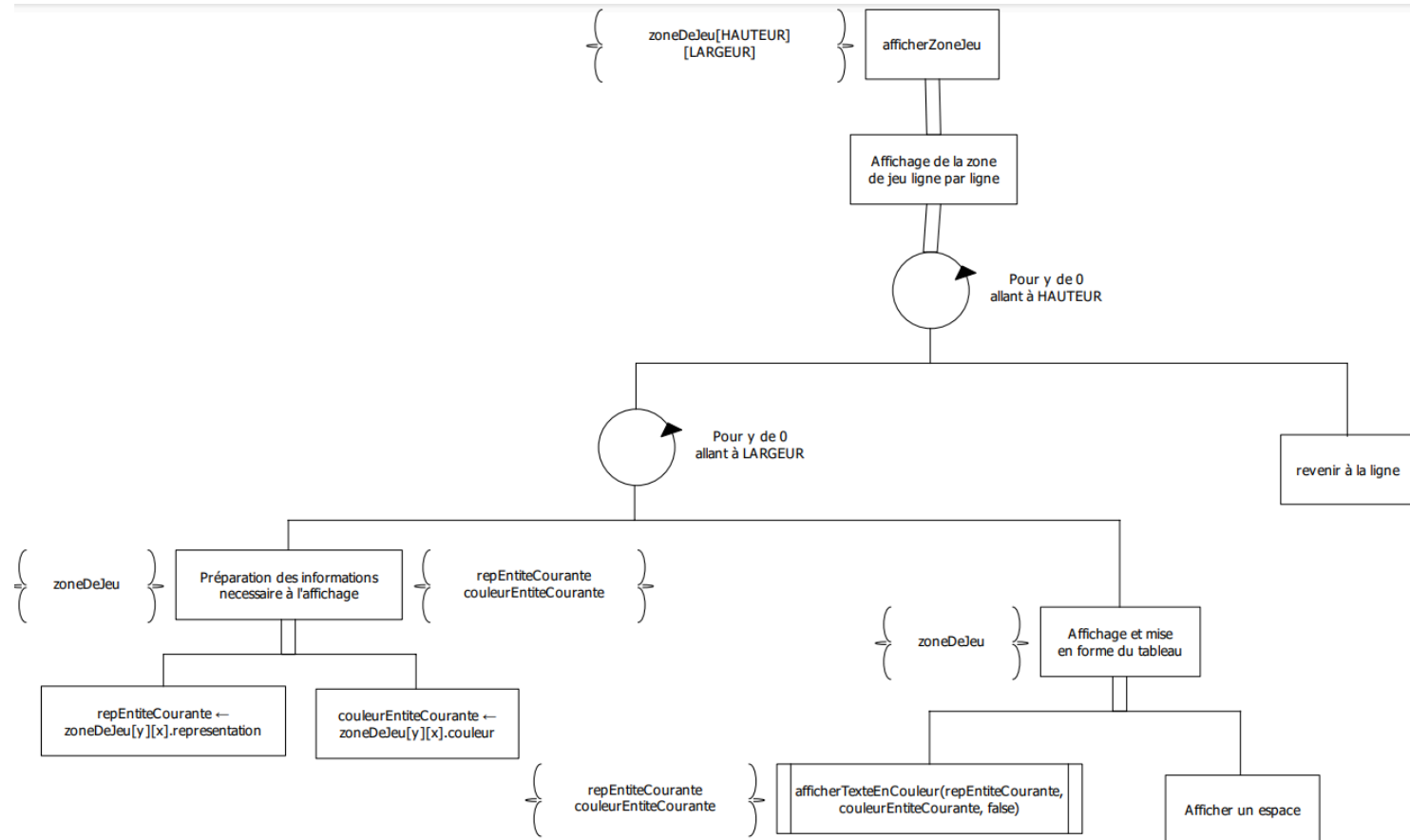
A. But de l'action

Afficher la zone de jeu ligne par ligne.

B. Stratégie de l'algorithme mise en œuvre et justification

L'algorithme est conforme à un parcours séquentiel complet avec traitement systématique. On cherche ici donc à afficher case par case et ligne par ligne le tableau contenant la zone de jeu.

## C. Algorithmes



D. Dictionnaire des éléments associés à cet algorithme

E. Dictionnaire des actions (complexes) associées à cet algorithme

#### 4.2.2 - generer

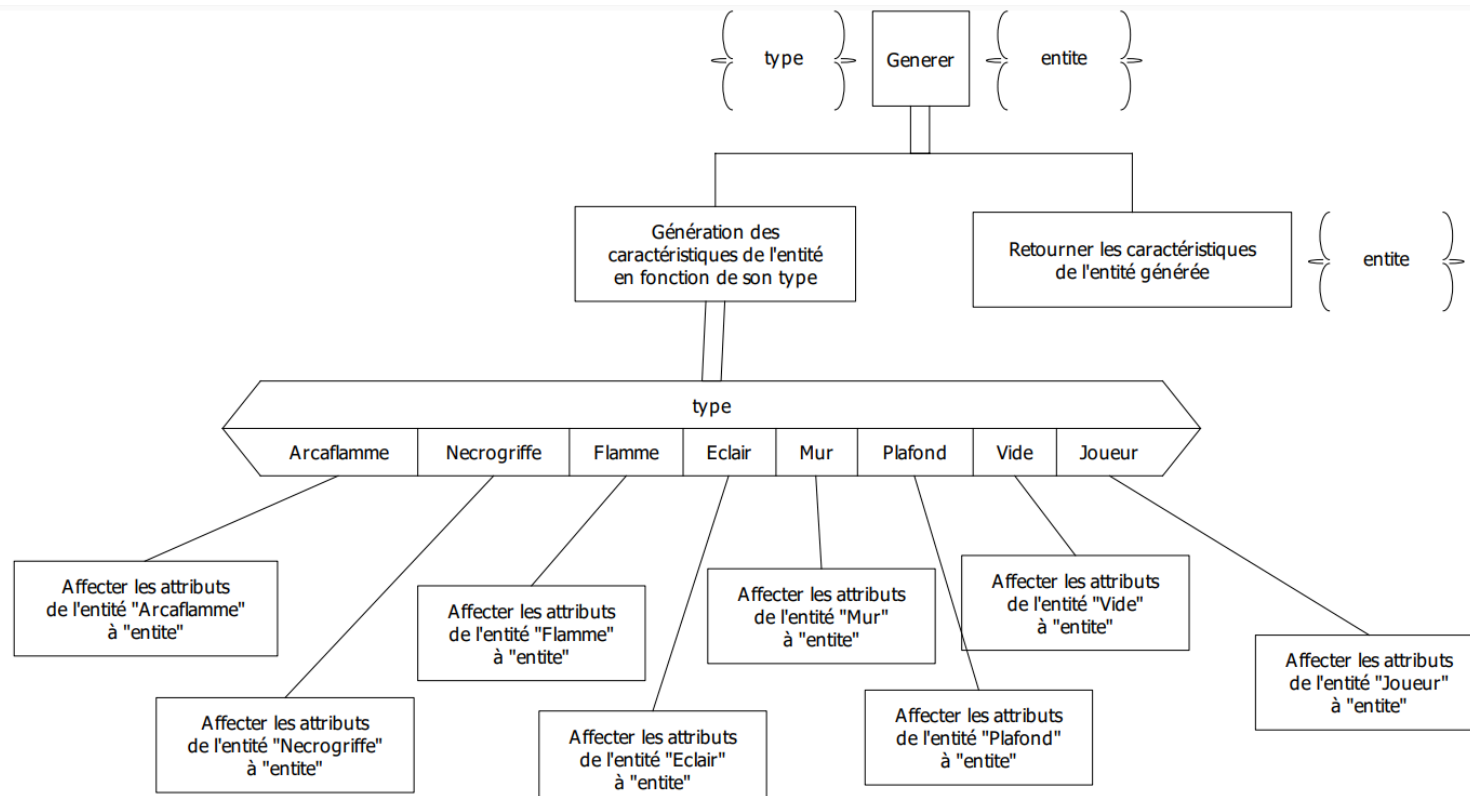
##### A. But de l'action

Générer les caractéristiques de l'entité "type" en paramètre.

##### B. Stratégie de l'algorithme mise en œuvre et justification

L'algorithme est conforme à aucun modèle d'algorithme vu en cours. On cherche ici à donner des caractéristiques à l'entité de type "type" en fonction de ce type.

## C. Algorithmes



#### D. Dictionnaire des éléments associés à cet algorithme

| Nom         | Type   | Signification                        |
|-------------|--------|--------------------------------------|
| Arcaflamme  | Entite | Donnée représentant les aracaflames  |
| Joueur      | Entite | Donnée représentant le joueur        |
| Necrogriffe | Entite | Donnée représentant les necrogriffes |
| Flamme      | Entite | Donnée représentant les flammes      |
| Eclair      | Entite | Donnée représentant les eclairs      |
| Mur         | Entite | Donnée représentant les mur          |
| Plafond     | Entite | Donnée représentant les plafonds     |
| Vide        | Entite | Donnée représentant les cases vides  |

#### E. Dictionnaire des actions (complexes) associées à cet algorithme

Aucune action complexe dans cet algorithme.

#### 4.2.3 - avoirCoordonnees

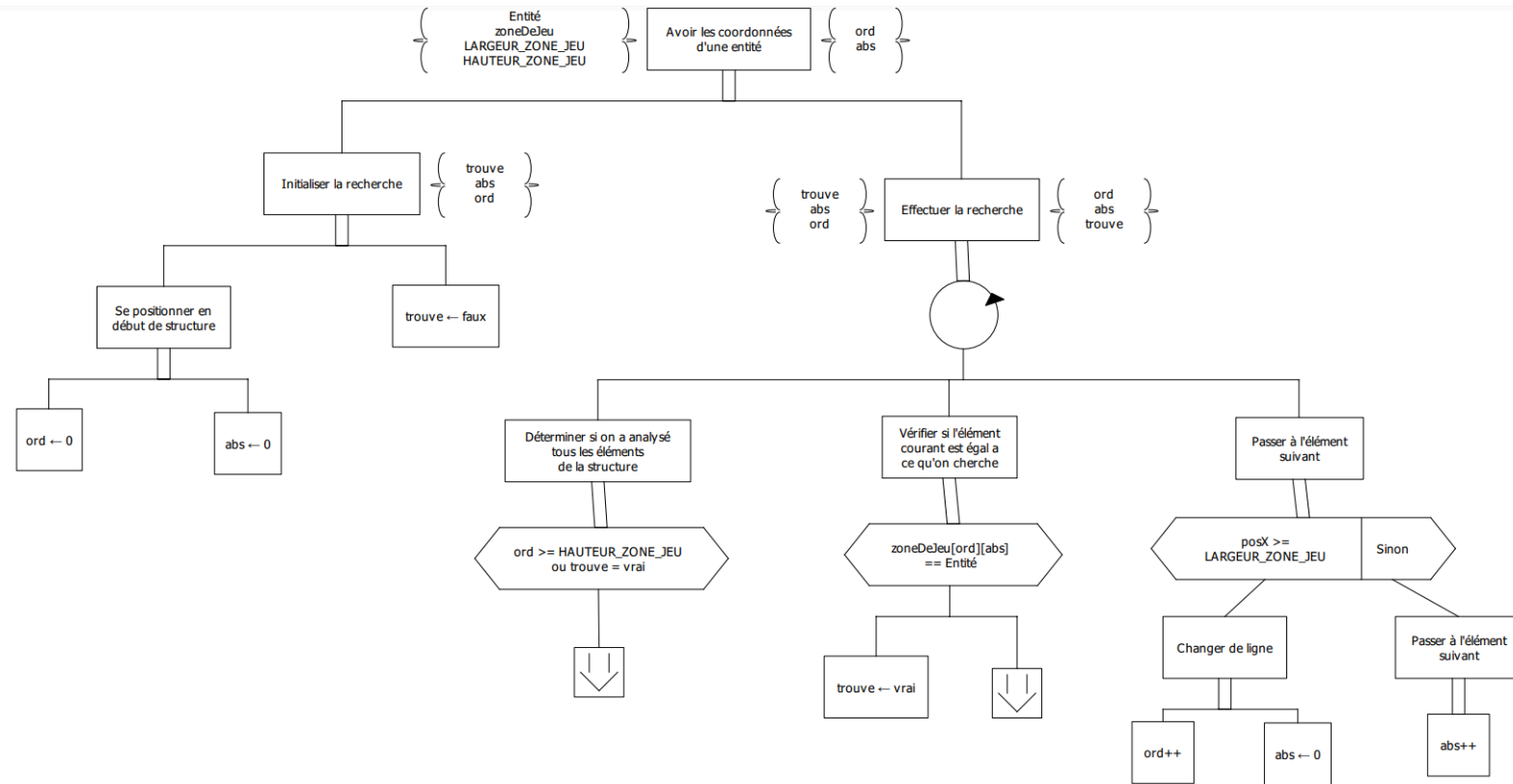
##### A. But de l'action

Dans le cas où l'entité de "type" existe dans la zone de jeu, renvoie ses coordonnées et trouve = vrai

##### B. Stratégie de l'algorithme mise en œuvre et justification

L'algorithme est conforme à une recherche séquentielle de première occurrence dans un tableau. On cherche le premier élément qui vérifie la condition `zoneDeJeu[ord][abs] == Entite`.

## C. Algorithmes



D. Dictionnaire des éléments associés à cet algorithme

E. Dictionnaire des actions (complexes) associées à cet algorithme

#### 4.2.4 - disparaitre(coordonnées)

##### A. But de l'action

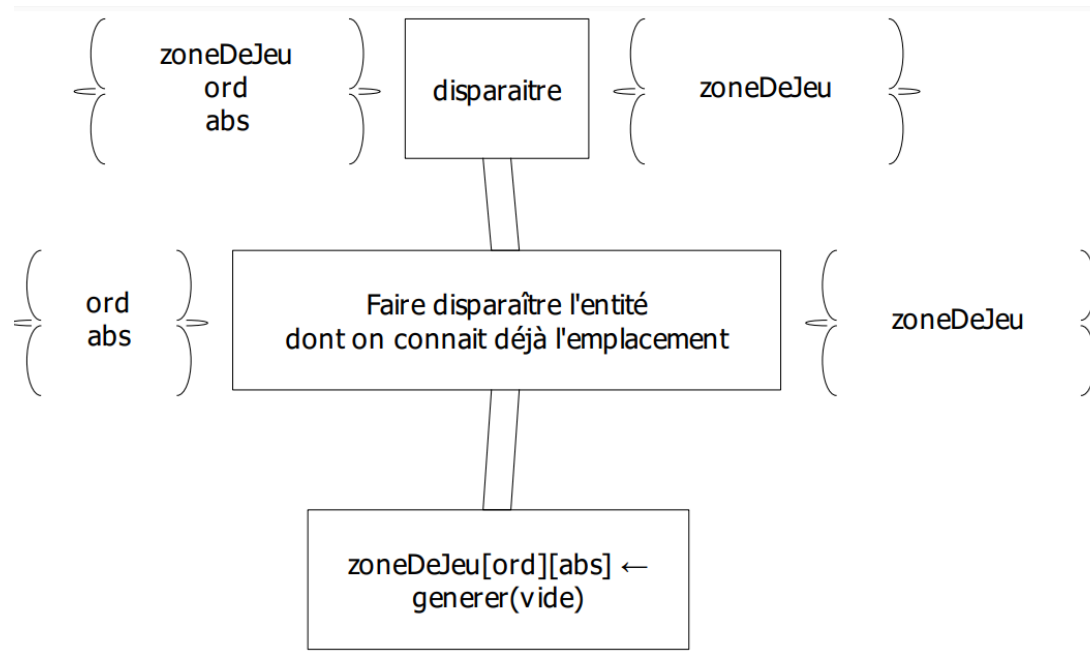
Remplacer, dans la zone de jeu, l'entité aux coordonnées d'abscisses et d'ordonnées définies par l'entité vide". Utilisé dans le cas où on connaît les coordonnées de l'entité.

##### B. Stratégie de l'algorithme mise en œuvre et justification

L'algorithme est conforme à un accès direct à la zone de jeu. On cherche à générer une case "vide" à l'emplacement indiqué.



### C. Algorithmes



### D. Dictionnaire des éléments associés à cet algorithme

| Nom       | Type             | Signification                              |
|-----------|------------------|--------------------------------------------|
| Vide      | Entite           | Donnée représentant les cases vides        |
| zoneDeJeu | Entite           | Tableau contenant tous les éléments du jeu |
| ord       | entier non signé | Ordonnée d'une entité le tableau zoneDeJeu |
| abs       | entier non signé | Abscise d'une entité le tableau zoneDeJeu  |

E. Dictionnaire des actions (complexes) associées à cet algorithme

Pas d'actions complexes dans cet algorithme.

#### 4.2.5 - disparaitre(TypeEntite)

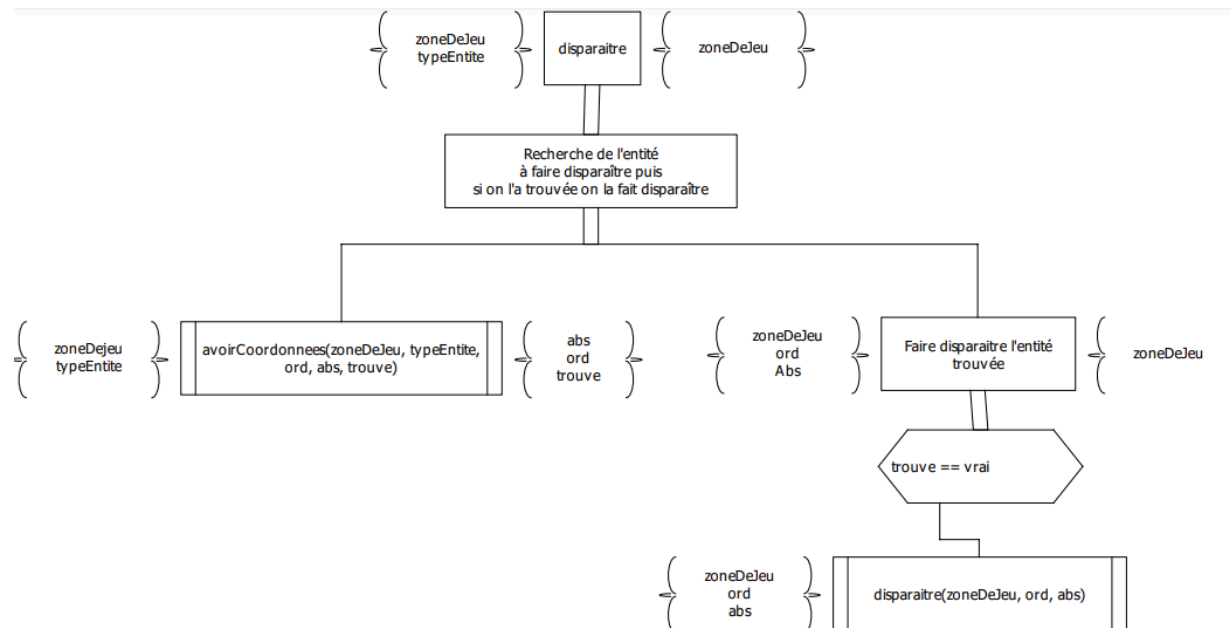
##### A. But de l'action

Remplacer, dans la zone de jeu, l'entité aux coordonnées d'abscisses et d'ordonnées définies par l'entité "vide". Utilisé dans le cas où on ne connaît pas les coordonnées de l'entité.

##### B. Stratégie de l'algorithme mise en œuvre et justification

L'algorithme est conforme à une recherche séquentielle puis à un accès direct de la zone de jeu. On cherche à connaître les coordonnées de l'entité que l'on souhaite faire disparaître, et ensuite on accède directement à sa position (s'il existe) et on le fait disparaître en le remplaçant par "vide".

##### C. Algorithmes



## Dictionnaire des éléments associés à cet algorithme

| Nom         | Type   | Signification                              |
|-------------|--------|--------------------------------------------|
| Arcaflamme  | Entite | Donnée représentant les aracaflammes       |
| Joueur      | Entite | Donnée représentant le joueur              |
| Necrogriffe | Entite | Donnée représentant les necrogriffes       |
| Flamme      | Entite | Donnée représentant les flammes            |
| Eclair      | Entite | Donnée représentant les eclairs            |
| Mur         | Entite | Donnée représentant les mur                |
| Plafond     | Entite | Donnée représentant les plafonds           |
| Vide        | Entite | Donnée représentant les cases vides        |
| zoneDeJeu   | Entite | Tableau contenant tous les éléments du jeu |

## F. Dictionnaire des actions (complexes) associées à cet algorithme

| Nom Action       | But                                                                                                                                                                              | Données                                                                    | Résultats                                           |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-----------------------------------------------------|
| avoirCoordonnees | Dans le cas où l'entité de "type" existe dans la zone de jeu, renvoie ses coordonnées et trouve = vrai                                                                           | const Entite<br>zoneDeJeu[HAUTEUR][LARGEUR]<br>TypeEntite typeEntite       | unsigned int ord<br>unsigned int abs<br>bool trouve |
| disparaitre      | Remplacer, dans la zone de jeu, l'entité aux coordonnées d'abscisses et d'ordonnées définies par l'entité "vide". Utilisé dans le cas où on connaît les coordonnées de l'entité. | Entite zoneDeJeu[HAUTEUR][LARGEUR]<br>unsigned int abs<br>unsigned int ord | Entite zoneDeJeu[HAUTEUR][LARGEUR]                  |

#### 4.2.4 - saisieUtilisateur

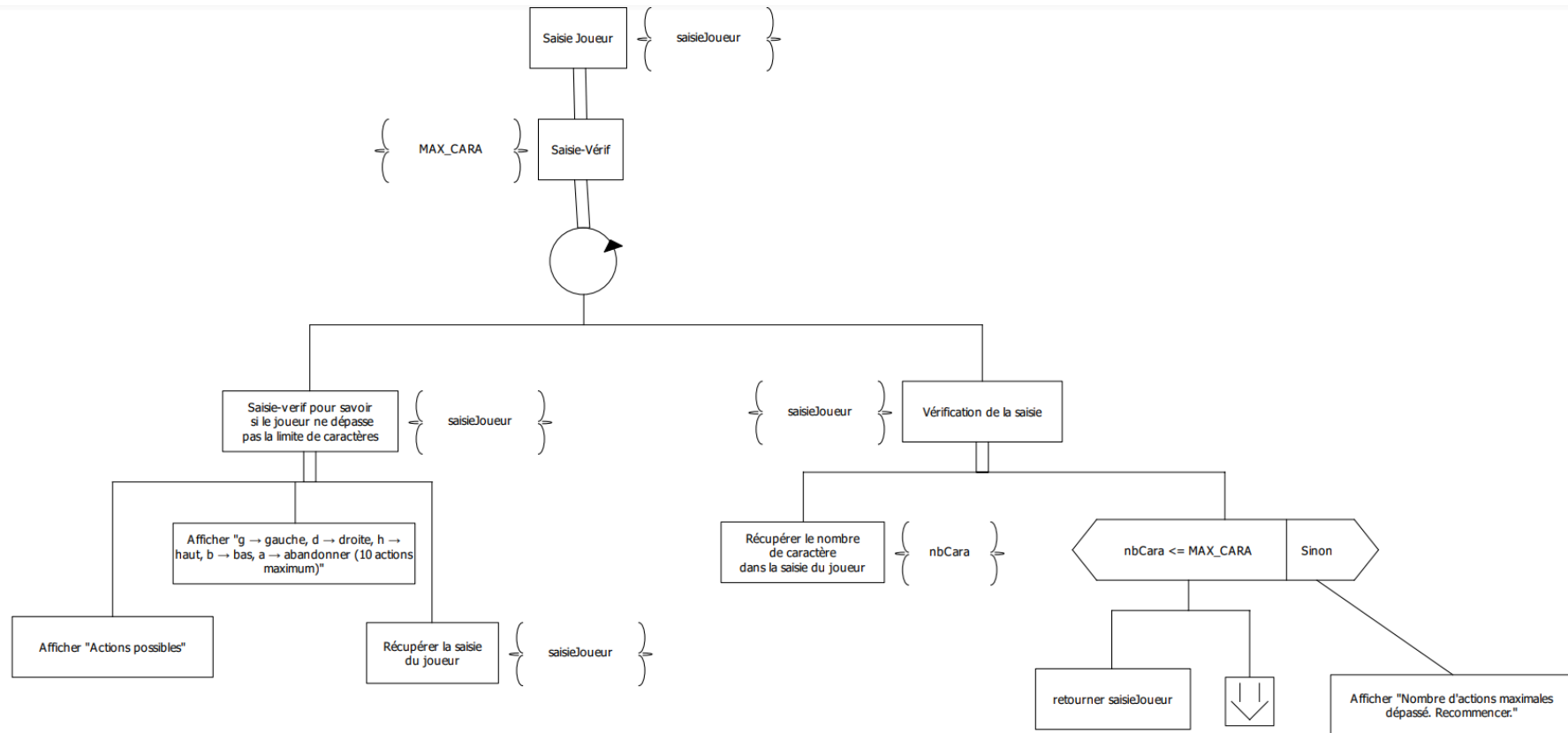
##### A. But de l'action

Envoyer une saisie à l'utilisateur avec un nombre de caractères limités et renvoyer cette dernière sous forme de chaîne de caractères (string)

##### B. Stratégie de l'algorithme mise en œuvre et justification

L'algorithme est conforme à une saisie-vérif. On cherche ici à inviter l'utilisateur à saisir les instruction, puis on vérifie si la saisie ne dépasse pas la limite de caractères imposés.

## C. Algorithmes



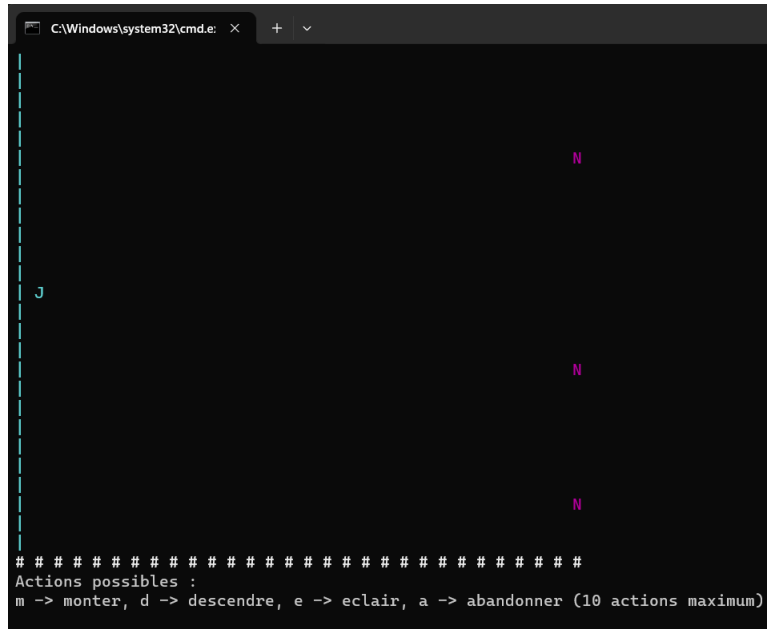
D. Dictionnaire des éléments associés à cet algorithme

| Nom      | Type             | Signification                                            |
|----------|------------------|----------------------------------------------------------|
| MAX_CARA | entier non signé | Nombre de caractères maximales que le joueur peut entrer |

E. Dictionnaire des actions (complexes) associées à cet algorithme

Il n'y a aucune action complexe associées à cet algorithme.

## 5. Tests réalisés et traces d'exécution



Les résultats obtenus sont consignés sur le tableau des jeux d'essais ci-dessous. Le problème de “zone de jeu coupée” est appuyé par la capture d'écran ci-contre où le plafond n'est pas affiché.

Ainsi, il est recommandé pour la manipulation des valeurs de respecter cette plage de valeurs:

HAUTEUR / LARGEUR → Entre 10 et 20

NOMBRE\_ENNEMIS → Entre 2 et 4

|   | A                        | B       | C       | D | E                               | F                 | G | H                                                                                              | I                                                                                     | J                                                         |
|---|--------------------------|---------|---------|---|---------------------------------|-------------------|---|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------|
| 1 | Tests programme Spellwar |         |         |   |                                 |                   |   | Résultats d'exécution                                                                          |                                                                                       |                                                           |
| 2 | Données                  |         |         |   | Résultats attendus              | Affichage écran   |   | Résultats obtenus                                                                              | Remarques - problèmes rencontrés - idées de solution                                  | Affichage écran                                           |
| 3 | NOMBRE_ENNEMIS           | HAUTEUR | LARGEUR |   | /                               | /                 |   | /                                                                                              |                                                                                       |                                                           |
| 4 | 3                        | 15      | 15      |   | Le jeu se comporte normalement. | Rien d'anormal    |   | Le jeu se comporte normalement                                                                 | OK                                                                                    | Rien d'anormal                                            |
| 5 | 2                        | 20      | 20      |   | Le jeu se comporte normalement. | Rien d'anormal    |   | Le jeu se comporte normalement                                                                 | OK                                                                                    | Rien d'anormal                                            |
| 6 | 4                        | 30      | 30      |   | Le jeu se comporte normalement. | Rien d'anormal    |   | La zone de jeu ne s'affiche pas entièrement                                                    | Limiter la HAUTEUR et la LARGEUR à 25 pour assurer le bon déroulement du jeu          | Zone de jeu coupée                                        |
| 7 | 5                        | 10      | 10      |   | Le jeu se comporte normalement. | Rien d'anormal    |   | Le jeu se comporte normalement, bien que ce soit difficilement jouable dû au surplus d'ennemis | Limiter le nombre d'ennemis à 4 pour éviter le surplus d'infos                        | Trop d'informations si on joue un peu, devient impossible |
| 8 | 6                        | 5       | 5       |   | Le jeu ne se lance pas.         | Rien ne s'affiche |   | Le jeu charge à l'infini                                                                       | Limiter le fait de pouvoir mettre un nombre d'ennemis < à la taille de la zone de jeu | Rien                                                      |
| 9 | 6                        | 15      | 15      |   | Le jeu se comporte normalement. | Rien d'anormal    |   | Le jeu se comporte normalement, bien que ce soit difficilement jouable dû au surplus d'ennemis | Limiter le nombre d'ennemis à 4 pour éviter le surplus d'infos                        | Trop d'informations si on joue un peu, devient impossible |



## 6. Remarques

Nous avons rencontré énormément de difficultés lors de cette SAÉ. En effet, de nombreux problèmes que nous n'avions jamais rencontré auparavant se sont présentés à nous, comme par exemple lors de la création du type Entite: Nous avons eu énormément de problèmes au niveau de la définition de ce type, et nous avons finalement fait le choix de le faire via une fonction (générer()).

Bien que nous ayons, nous pensons, passé toutes les séances de SAÉ (en autonomie comme qu'avec le professeur) à les exploiter à leur plein potentiel, en nous concentrant durant ces séances et en posant de temps en temps des questions, nous avons eu beaucoup de mal à terminer le travail. C'est d'ailleurs le professeur qui nous a donné l'idée de définir le type Entite dans une fonction au lieu de le définir directement.

Cependant, nous estimons avoir travaillé en dehors des SAÉ plus d'une trentaine d'heures, ce que nous trouvons assez conséquent.

Cependant nous sommes assez satisfaits de la répartition du travail qui nous a semblé assez équitable

## 7. Code C++

### 7.1 - Choix d'organisation des fichiers composant le code source

Le code a été organisé de façon à ne tenir que dans un seul module, ici module.cpp.

Nous avons fait ce choix afin d'éviter de nous emmêler les pinceaux avec une trop grosse quantité de fichier .cpp et .h.

Voici la liste des sous-programmes présents dans le module.cpp:

```
Entite generer(TypeEntite type);

void initialiserZoneDeJeu(Entite zoneDeJeu[HAUTEUR][LARGEUR]);

void initialisationEntites(Entite zoneDeJeu[HAUTEUR][LARGEUR]);

void afficherZoneJeu(const Entite zoneDeJeu[HAUTEUR][LARGEUR]);

void actionJoueur(string deplacement, Entite zoneDeJeu[HAUTEUR][LARGEUR], unsigned short int &indiceLettreLue);

void avoirCoordonnees(const Entite zoneDeJeu[HAUTEUR][LARGEUR], TypeEntite typeEntite, unsigned int &ord, unsigned int &abs, bool &trouve);

string saisieUtilisateur(void);

void disparaître(Entite zoneDeJeu[HAUTEUR][LARGEUR], TypeEntite typeEntite);

void disparaître(Entite zoneDeJeu[HAUTEUR][LARGEUR], unsigned int abs, unsigned int ord);

char verifFinDePartie(const Entite zoneDeJeu[HAUTEUR][LARGEUR]);
```

```

void actionsEntite(Entite zoneDeJeu[HAUTEUR][LARGEUR]);

void genererActionEntite(Entite zoneDeJeu[HAUTEUR][LARGEUR], TypeEntite typeEntite,
                        unsigned int &ord, unsigned int &abs);

char gererCollisionsNecrogriffe(Entite zoneDeJeu[HAUTEUR][LARGEUR], unsigned int futurOrd, unsigned int futurAbs);

char gererCollisionsFlamme(Entite zoneDeJeu[HAUTEUR][LARGEUR], unsigned int actuOrd, unsigned int actuAbs,
                           unsigned int futurOrd, unsigned int futurAbs);

char gererCollisionsJoueur(Entite zoneDeJeu[HAUTEUR][LARGEUR], unsigned int futurOrd, unsigned int futurAbs);

char gererCollisionsEclair(Entite zoneDeJeu[HAUTEUR][LARGEUR], unsigned short int actuOrd, unsigned short int actuAbs,
                           unsigned short int futurOrd, unsigned short int futurAbs);

char gererCollisionsArcaflamme(Entite zoneDeJeu[HAUTEUR][LARGEUR], unsigned int futurOrd, unsigned int futurAbs);

void avancerProjectiles(Entite zoneDeJeu[HAUTEUR][LARGEUR], TypeEntite typeEntite, unsigned int ord, unsigned int abs);

void nouveauTourJoue(Entite zoneDeJeu[HAUTEUR][LARGEUR]);

void switchCollisionsNecrogriffe(Entite zoneDeJeu[HAUTEUR][LARGEUR], unsigned int actuOrd, unsigned int actuAbs,
                                unsigned int futurOrd, unsigned int futurAbs);

void switchCollisionsArcaflamme(Entite zoneDeJeu[HAUTEUR][LARGEUR], unsigned int actuOrd, unsigned int actuAbs,

```

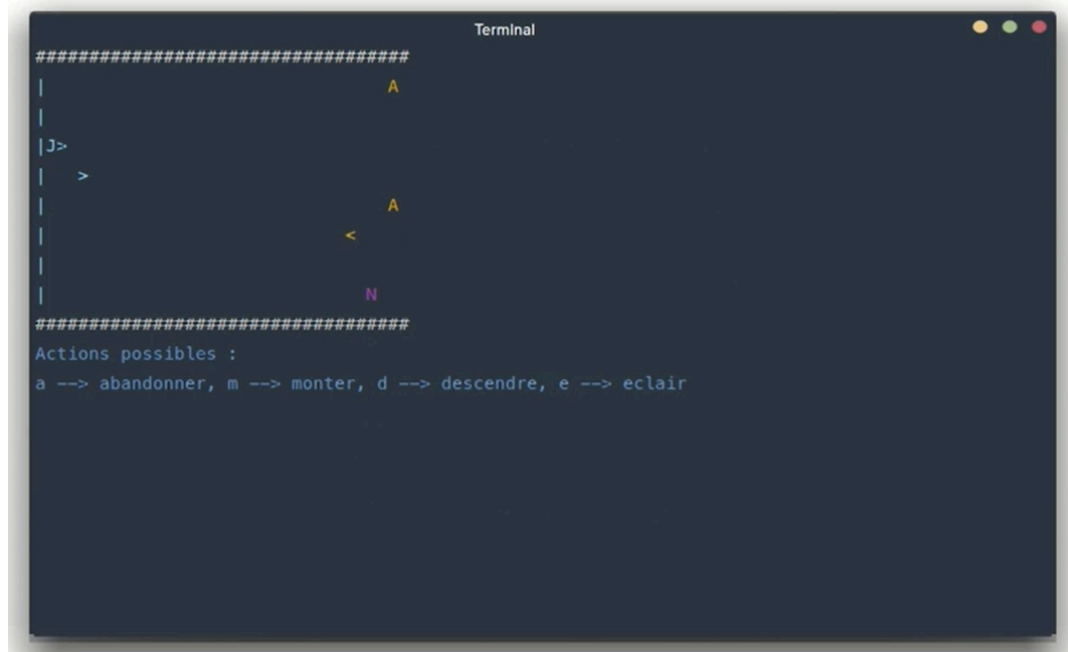
```
        unsigned int futurOrd, unsigned int futurAbs);  
  
void switchCollisionsFlamme(Entite zoneDeJeu[HAUTEUR][LARGEUR], unsigned int actuOrd, unsigned int actuAbs,  
        unsigned int futurOrd, unsigned int futurAbs);  
  
void switchCollisionsEclair(Entite zoneDeJeu[HAUTEUR][LARGEUR], unsigned int actuOrd, unsigned int actuAbs,  
        unsigned int futurOrd, unsigned int futurAbs);  
  
void switchCollisionsJoueur(Entite zoneDeJeu[HAUTEUR][LARGEUR], unsigned int actuOrd, unsigned int actuAbs,  
        unsigned int futurOrd, unsigned int futurAbs);
```

## 7.2 - Code source

***Les fichiers .cpp et .h ne sont pas inclus dans le présent document, mais sont joints au dossier avec un paragraphe en-tête certifiant l'originalité du code produit***

## 8. Annexes

### Annexe 1 – maquettes d'écran prévues dans les spécifications



```
#####
|                                     A
|
|J>
|  >
|
|                                     A
|                                <
|
|                                     N
|
#####
Actions possibles :
a --> abandonner, m --> monter, d --> descendre, e --> eclair
```

## Annexe 2 – Traces d'exécution - copies d'écran

```
# # # # # # # # # # # # # # #  
|  
  
    >  
    >      >  
J >          < < < < < A  
  
  
                N   < A  
            <  
  
# # # # # # # # # # # # # # #  
Actions possibles :  
m -> monter, d -> descendre, e -> eclair, a -> abandonner (10 actions maximum)  
|
```